ALOUANI Fadi

BTS SIO 1
SLAM
02/10/26

TD : exercices pratiques Supabase & Flutter

Sommaire

Introduction....1
Niveau 1 — Découverte....2
Niveau 2 — Relationnel....4
Niveau 3 — Sécurisation des données avec l'authentification....5
Conclusion....6

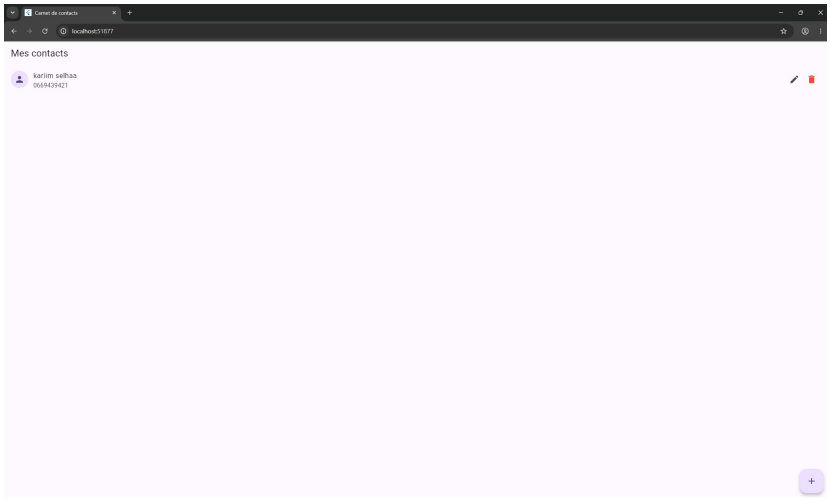
Introduction

Dans ce TP, j'ai découvert Supabase et son utilisation avec Flutter. L'objectif était de créer une application simple permettant de gérer des contacts avec une base de données PostgreSQL.

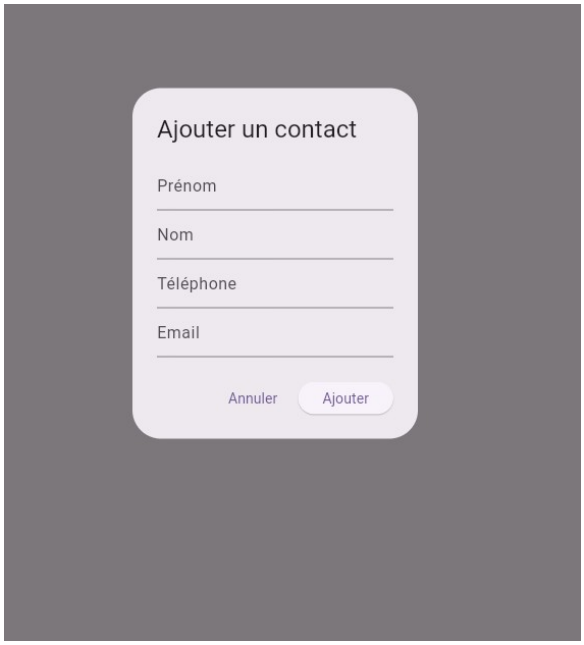
Niveau 1 — Découverte

Pour commencer, j'ai créé un projet Flutter nommé carnet_contacts et je l'ai connecté à mon projet Supabase grâce au package supabase_flutter.
J'ai ensuite créé une table contacts dans ma base de données PostgreSQL. Cette table permet de stocker différentes informations sur les contacts, comme le nom, le prénom, le téléphone et l'adresse e-mail.
J'ai développé les principales opérations CRUD dans mon application :

- CREATE : j'ai ajouté un formulaire permettant de créer un nouveau contact.
- READ : j'ai récupéré les contacts depuis Supabase et je les ai affichés dans l'application.
- UPDATE : j'ai ajouté la possibilité de modifier les informations d'un contact.
- DELETE : j'ai ajouté un bouton permettant de supprimer un contact après confirmation.



Ajouter un contact :



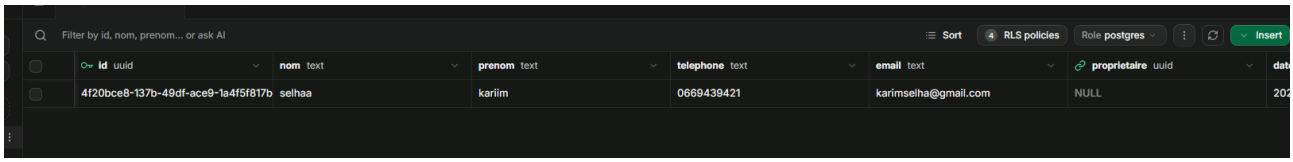
Modifier le contact :



Supprimer le contact :

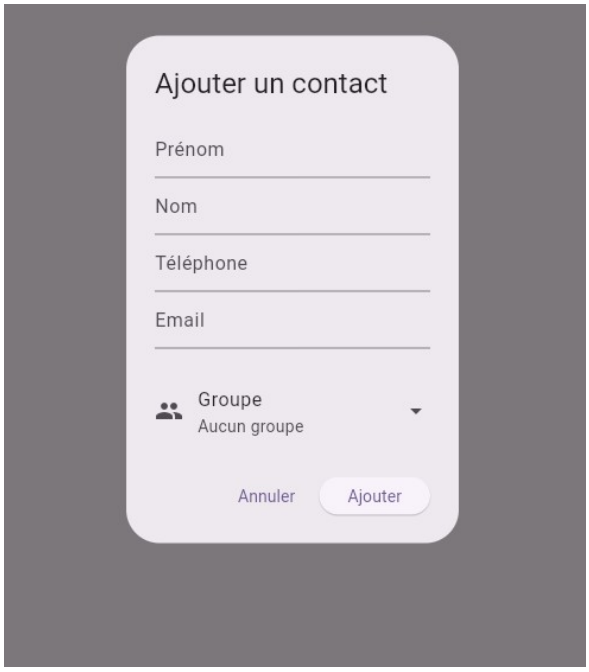


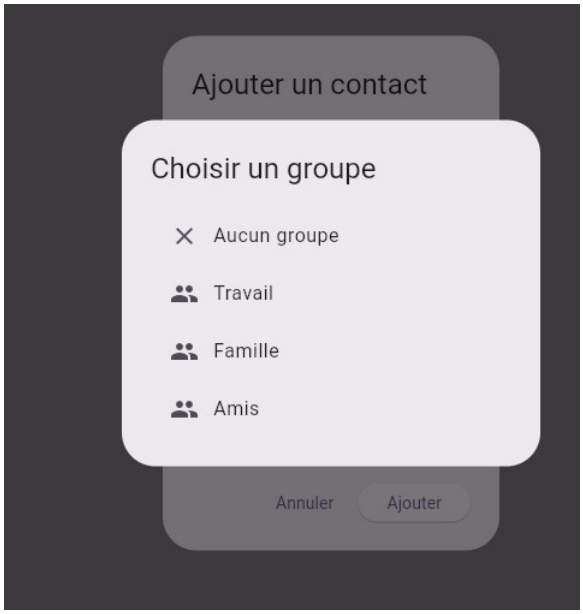
Le contact apparaît bien dans la bdd



Niveau 2 — Relationnel

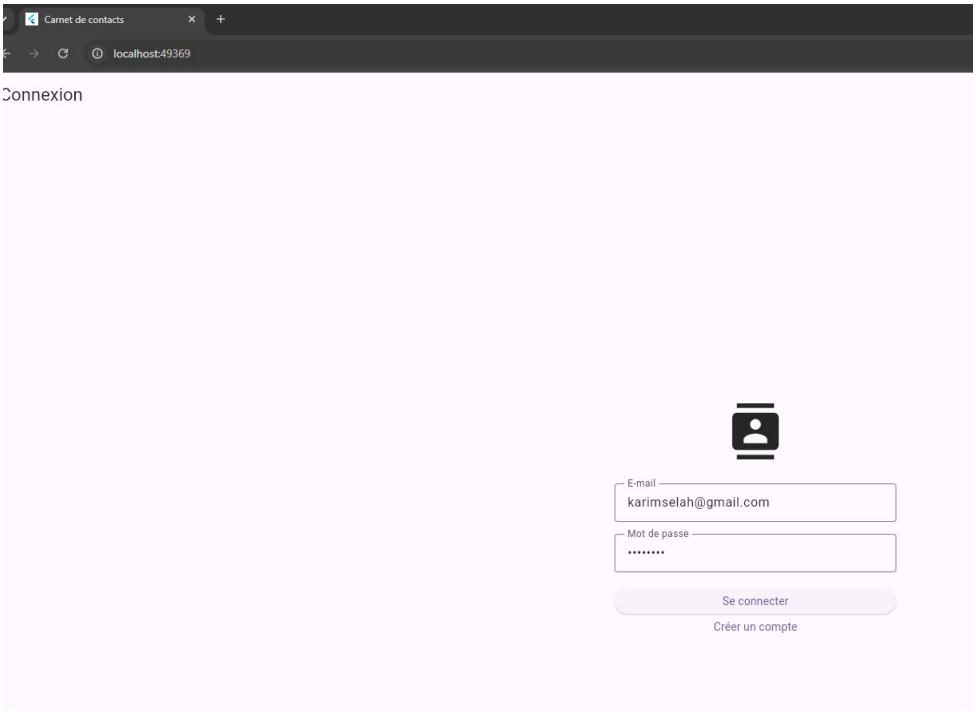
Dans cette deuxième partie, j'ai approfondi l'utilisation de PostgreSQL avec Supabase en travaillant sur les relations entre plusieurs tables.
J'ai commencé par créer une nouvelle table groupes contenant différents groupes comme « Amis », « Famille » et « Travail ». J'ai ensuite ajouté une clé étrangère groupe_id dans la table contacts afin de pouvoir associer chaque contact à un groupe.
J'ai modifié mon application Flutter pour récupérer les groupes depuis Supabase et permettre à l'utilisateur de choisir un groupe lorsqu'il ajoute ou modifie un contact.
J'ai également utilisé une requête relationnelle pour récupérer les informations du groupe associé à chaque contact. Le nom du groupe est ensuite affiché directement dans la liste des contacts.
Enfin, j'ai configuré une politique RLS permettant à l'application de lire les groupes depuis Supabase.





Niveau 3 — Sécurisation des données avec l'authentification

J'ai d'abord utilisé l'authentification de Supabase pour permettre aux utilisateurs de se connecter à leur compte avec une adresse e-mail et un mot de passe.
J'ai ensuite ajouté une colonne proprietaire dans ma table contacts. Cette colonne permet d'associer chaque contact à l'utilisateur qui l'a créé.
Dans mon application Flutter, lors de la création d'un contact, je récupère automatiquement l'identifiant de l'utilisateur connecté et je l'enregistre dans la colonne proprietaire.
J'ai également configuré les politiques Row Level Security (RLS) sur la table contacts. Ces règles permettent de contrôler les opérations effectuées sur les données.
Ainsi, un utilisateur connecté peut uniquement consulter, ajouter, modifier et supprimer ses propres contacts.
J'ai également conservé les fonctionnalités CRUD réalisées précédemment : création, lecture, modification et suppression des contacts.



Niveau 4 — Amélioration de l’application

Niveau 5 — Final

Conclusion

Ce premier exercice m'a permis de comprendre comment connecter une application Flutter à Supabase et comment utiliser une base PostgreSQL depuis l'application. J'ai également découvert le fonctionnement du CRUD et des politiques de sécurité RLS. Cette première étape me servira de base pour utiliser les fonctionnalités plus avancées de Supabase dans la suite du TP.